

TACO: A Tiled Architecture of (re)Configurable Operators

Jason Yan (zexuany2), Nitish Bhupathi Raju (nbhup2),
Vinit Gupta (vinitg2), Feiyang Liu (fl22), Rudra Thakkar (rudrapt2)

ECE 427 Advanced VLSI System Design
Final Project Report

Abstract—In this project, our group designed an FPGA with a custom architecture in the TSMC 65nm process. Our FPGA fabric features Configurable Logic Blocks (CLBs) that form the core of the reconfigurable logic array. The routing network consists of Switch Boxes (SBs) with 8-wide routing tracks, enabling programmable interconnections between CLBs, and Connection Boxes (CBs) that interface each CLB to the surrounding routing fabric. At the center of the FPGA fabric, we integrate dedicated Multiply-and-Accumulate (MAC) units to support arithmetic-intensive applications. Surrounding the core fabric, the FPGA includes 1 KB of on-chip memory for data storage, GPIO pins for external interfacing, a clock divider that dynamically adjusts the operating frequency based on the supported frequency ($F_{\max}$) of the mapped design, and a JTAG interface used for programming and debugging by shifting in bitstreams. To support the full design flow, we utilize the open-source Verilog-to-Routing (VTR) framework to compile Verilog designs into bitstreams compatible with our FPGA architecture. Furthermore, we designed a custom printed circuit board (PCB) to facilitate post-silicon validation. We successfully demonstrated chip interaction and programming via OpenOCD, and validated end-to-end functionality by running mapped circuits, including an ALU utilizing physical switches and LEDs for I/O.

I. PROJECT OVERVIEW

The general goal of this project is to design, verify, tape out, and validate a custom-designed FPGA in the TSMC 65nm process with custom compiler support. By the end of the project, we implemented:

- 1) 288 Configurable Logic Blocks (CLBs), each with a single 4-input lookup table, a flip-flop, and a mux.
- 2) An 8-track routing network between CLBs through connection blocks and switch blocks.
- 3) JTAG hardware for programming and debugging.
- 4) 39 GPIO pins for I/O.
- 5) A Verilog-to-bitstream compiler based on open-source projects.
- 6) 64 selectable clock frequencies through clock dividers.
- 7) 1 KB BRAM (Memory).
- 8) 9 Multiply-and-Accumulate Units (MACs).
- 9) A custom PCB for chip bring-up, enabling successful bitstream programming via OpenOCD and hardware execution of circuits like an ALU.

II. MAIN FEATURES AND FUNCTIONALITY

Our FPGA can run RTL designs (that fit within our resource budget) at a programmable frequency (up to the $F_{\max}$ of the design). To aid in versatility, the I/O pins are GPIOs that can be programmed through an independent bitstream, allowing reconfigurability based on the RTL design's needs. Furthermore, on-chip BRAM and on-fabric MAC units are available to accelerate data storage and arithmetic computations, respectively. We also created an end-to-end Verilog-to-bitstream compiler for our FPGA architecture to facilitate ease of use.

A. Design Choices and Justification

One major design choice was the size of the CLB array. We created a parameterizable design and explored various values until we found the maximum size that fit within the area constraints, which turned out to be 18×17 . Additionally, we wanted to heavily prioritize reconfigurability in every part of the chip. This motivated the choice to add GPIO pins and a programmable clock divider. Furthermore, we decided to add support for JTAG to improve usability in real-world scenarios.

We also prioritized adding two advanced features to our chip: BRAM and MAC units. Most modern FPGAs contain on-chip memory; therefore, we decided to add some memory support to facilitate practical RTL designs that require data storage. Additionally, we integrated MAC units into the design to create a novel FPGA architecture capable of accelerating multiply-add computations. These units also serve as a proof of concept for incorporating arbitrary hard IPs into the fabric with compiler support.

B. Comparison with Proposal

We added most of the features that we were planning to implement from the project proposal (these features are described in the Project Overview section). However, the following advanced features were proposed but were not implemented due to the prioritization of other advanced features: a carry-chain for the CLBs, clustered CLBs, and an on-chip RISC-V core.

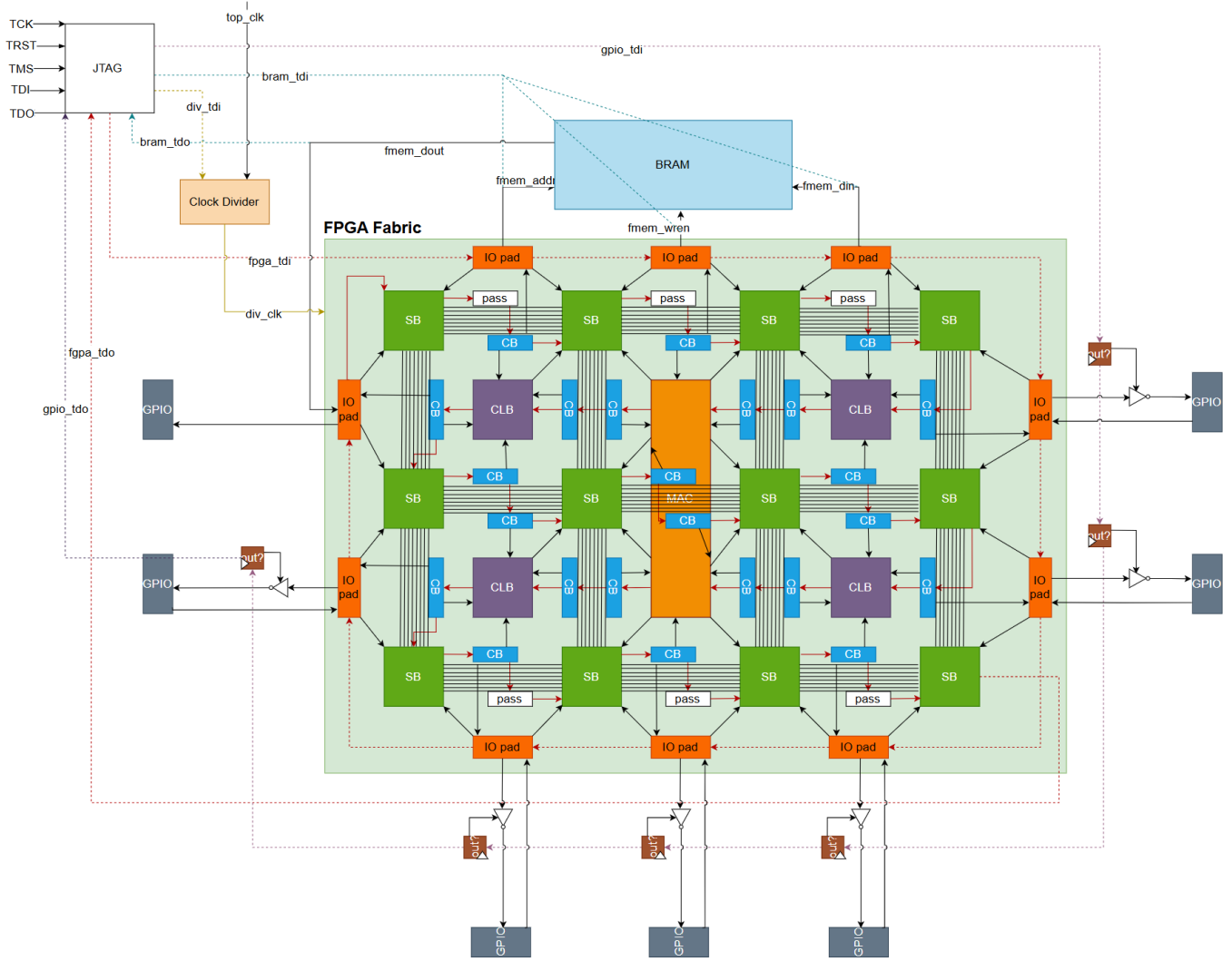


Fig. 1: High-Level Design Diagram

III. HIGH-LEVEL ARCHITECTURE

The high-level block diagram of our FPGA is shown in Figure 1. The figure includes a 2×3 fabric for simplicity. Note: Within the FPGA fabric, black lines indicate the interconnections, while the red lines indicate the scan-chain for JTAG.

IV. PHYSICAL DESIGN

Figure 2a shows the physical layout of TACO. We also decided to add some signatures to our chip on the AP layer. Figure 2b shows this view. Our chip has the following properties:

- 1) FPGA Clock Frequency: 20 MHz
- 2) JTAG Frequency: 10 MHz
- 3) Area Utilization: 85.756% for a $708 \times 708 \mu\text{m}^2$ core site
- 4) Power Estimation: 7.214 mW
- 5) Input/Output Pins:
 - a) 2 VDD pins
 - b) 2 VSS pins

- c) 8 pins that are always inputs
- d) 1 pin that is always an output
- e) 39 GPIO pins that can be programmed as inputs or outputs

V. DESIGN DOCUMENTATION

A. CLB Architecture

Our baseline CLB implementation contains one LUT4, a flip-flop (FF), and a mux to select between the combinational output from the LUT4 and the sequential output from the FF to determine if the CLB is modeling combinational or sequential logic. The CLB architecture is shown in Figure 3.

B. Connection Box (CB)

Each connection box is responsible for connecting the input of the CLB to the global routing network. Since each CLB has 4 inputs, each CLB has 4 CBs that surround it. Each CB chooses one of the eight tracks as the input to the CLB. When a CLB input is left unused, it is set to 0.

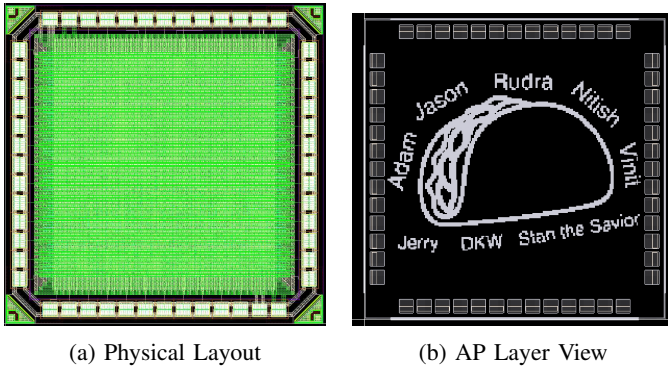


Fig. 2: Physical Design View

C. Switch Box (SB)

Each switch box is responsible for connecting tracks to create the desired routing. We used a unidirectional switch box structure that enables connections to three different tracks (up, down, and continue in the same direction). We implemented the universal switch box architecture as shown in Figure 4.

Certain SBs have "extra inputs" coming into the switch box to connect IO pad inputs, CLB outputs, and MAC outputs to the routing network. These extra inputs are fed into the fourth input of a 4:1 mux (which would otherwise be left unconnected).

D. Track Width

The number of tracks required per channel depends on the level of complexity. According to multiple studies, with around 200 single-LUT4 CLBs, the most complex design will require at least 6-7 tracks per channel. We received some suggestions to use 8 as our width to ensure that it will not be the limiting factor when mapping designs. Therefore, we elected to use 8 as our width.

E. BRAM

We implemented an off-fabric 1 KB BRAM that the FPGA fabric can communicate with using designated IO pads. The BRAM has the following ports: an 8-bit address, a 4-bit din, a 1-bit wen, and a 4-bit dout. During the design phase, we supported both SRAM- and register-file-based BRAM implementations, and ultimately selected the register-file implementation to enable smoother integration.

F. MAC

There is a column of MACs at the center of the FPGA fabric. Each MAC has the following inputs: 3-bit a, 3-bit b, 1-bit mac_enable, and 1-bit mac_clear. Each MAC has an 8-bit output. If the RTL logic mapped onto the fabric needs to use MAC, these signals need to be specifically named and used in the RTL.

G. Clock

For clocking, TACO can use an external clock source, such as a host FPGA, a function generator, or another external

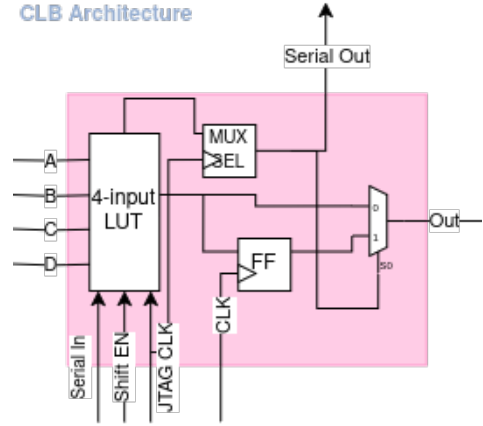


Fig. 3: CLB Architecture

oscillator. This clock is then passed through a standard divide-by-even clock divider, whose output is provided to the FPGA fabric.

H. Verilog-to-bitstream Generation

Our bitstream generation flow involves open-source tools, including Yosys and Versatile Place and Route (VPR). Yosys is a synthesis tool that takes Verilog designs and generates a netlist. VPR takes in the netlist, our FPGA architectural description file in XML, and some constraints, and generates packed netlists and the corresponding placement and routing on the FPGA. The generated files from VTR include several intermediate files, such as .net, .place, .route, .blif, and .fasm. These files are subsequently processed through custom Python scripts to generate the final bitstream for our FPGA. An example workflow from the OpenFPGA project is shown in Figure 5.

I. JTAG

The primary interface for programming the FPGA's configuration bitstream and for low-level debugging is the industry-standard IEEE 1149.1 JTAG. This serial interface minimizes pin count requirements for our design.

We made trivial modifications to an open-source JTAG implementation to create shift_enable signals for different instructions. We added four new instructions: FPGA_SHIFT_IR, CLOCK_DIV_IR, GPIO_IR, and MEM_JTAG_IR (which allows the JTAG controller to read from/write into BRAM). The first three instructions enable the shifting logic for the corresponding module. This way, each module of the chip is independently programmable.

J. User Data Interface

For general-purpose user interaction, the FPGA utilizes parallel General Purpose Input/Output (GPIO) pins. These pins can be configured by the loaded bitstream as either inputs or outputs, allowing the FPGA logic to directly interact with external components. The tri-state buffer logic to support GPIO is shown in Figure 1.

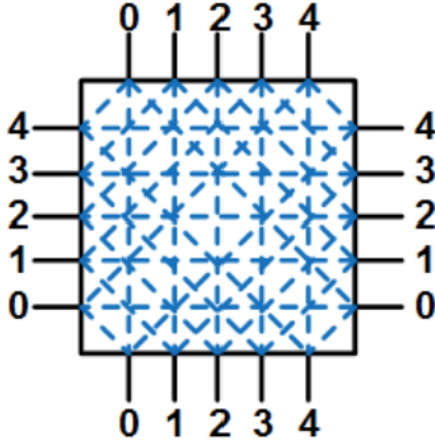


Fig. 4: Universal Switch Box

VI. VERIFICATION AND TEST STRATEGY

A. Verification Flow

The following verification strategy summarizes the features verified, the testing methodologies used, and the prioritization of verification phases. This strategy was adopted for both RTL and post-PnR verification.

a) Tile-Level Verification: Since the FPGA architecture consists of repeated tiles, verification begins at the tile level. Each tile includes a Configurable Logic Block (CLB) and routing units (connection and switch blocks). Simple synthesized standard cells (up to four inputs and one output) are tested on a small 2×2 FPGA to validate CLB functionality and routing correctness.

b) Functional Block-Level Verification: After tile-level testing, the FPGA is scaled to a 10×10 configuration for larger designs. Connectivity blocks are verified using constrained-random testing, with the intended RTL serving as the golden model. Additional components, such as the JTAG TAP controller, clock divider, and GPIOs, are verified individually before being integrated into the FPGA fabric.

c) Full-Chip Verification: Following block-level validation, the entire chip is tested as a single unit by toggling JTAG control pins, loading configuration bitstreams, and applying randomized GPIO inputs while monitoring outputs. Advanced features, such as the in-fabric MAC and memory units, are also verified.

d) Functional/Toggle Coverage: Each CLB is tested in both combinational and sequential modes. Larger designs, including ALUs and a simple processor, are synthesized and verified on the FPGA. Toggle coverage for RTL simulation is approximately 80% at a 14×15 fabric, limited by dead code in the open-source JTAG TAP controller, dummy logic from the tiled design, and unused IP functions that do not impact overall behavior.

e) Automated Regression Tool: A Python-based regression framework automates test generation, execution, and error reporting. Inputs and timing variations are randomized to enhance coverage and verification robustness.

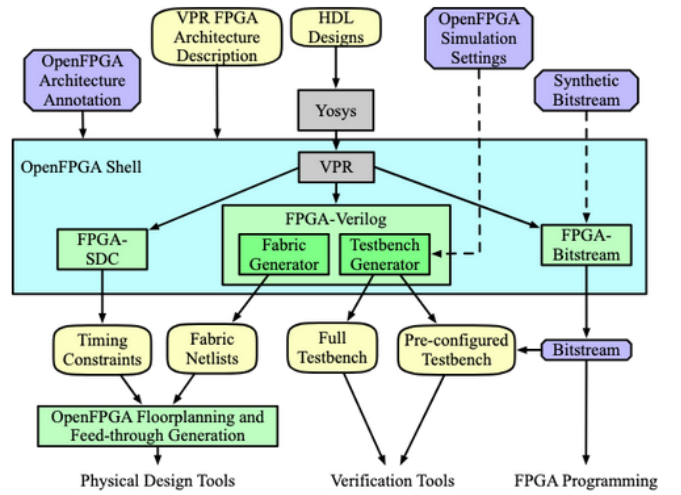


Fig. 5: Example Bitstream Generation Workflow

f) Post-PnR Verification: We modified the original testbench to drive the stimulus at the negative edge of the clock to ensure the set input delay is properly respected. We also utilized a testbench similar to the RTL simulation, but with path delays annotated by the SDF files generated by Innovus. The regression tool is applied to check correctness across all test cases.

B. DFT Features

Numerous DFT features were added to the design, and they are described in detail below.

a) Scan Chain: One benefit of our scan chain approach to programming is that we can easily use this feature for DFT. By shifting out the bitstream, we can retrieve the internal state of most flip-flops in the FPGA fabric, which allows us to verify the functionality of the design.

b) BRAM: One important feature of our design that is not included in the scan chain is BRAM. To enable testing, we decided to add a DFT feature where the contents of the BRAM can be written to and read from using JTAG.

c) Backup Shift Pins: To program any part of the chip, our initial plan was to use JTAG. However, this creates a single point of failure; if JTAG does not work as intended, we cannot control any aspect of the chip. To circumvent this, we dedicated two I/O pins to serve as a backup (Master Shift En and Master Shift In). We created a simple protocol in the chip where, once certain instructions are sent through these pins, all components of the chip can be programmed through them instead of JTAG.

VII. POST-SILICON VALIDATION PLAN

Note: This section outlines our initial post-silicon validation goals and proposed timeline, which were formulated prior to chip fabrication.

We planned to design a PCB to bring up and validate our chip. The chip was intended to be programmed using a standard FPGA development board (such as the Digilent

Genesys 2). For initial testing, we planned to drive JTAG pins directly from the FPGA by following the TAP controller state machine developed for the testbench framework. For more robust testing, we aimed to program the chip using OpenOCD as a driver for JTAG through a JTAG HS3 cable.

FPGA Dev-Board Programming: We planned to use the existing testbench framework (developed in SystemVerilog) to be synthesized on the FPGA as TACO's control unit.

Timeline:

- 01/15 - 02/15: PCB design, FPGA dev board setup + testing.
- 02/15 - 02/28 (chip arrival): Power + reset testing, scan chain shift-in/out.
- 03/01 - 03/21: Standard cell bitstream test.
- 03/21 - 04/15: Advanced bitstream test (FSM + CPU).
- 04/15 - 05/15: OpenOCD bring-up, debug + tests.

VIII. BRING-UP METHODOLOGY AND RESULTS

While the custom PCB design and manufacturing proceeded exactly according to plan, the physical chip bring-up phase experienced slight delays due to the late arrival of the fabricated chips. Despite these scheduling adjustments, the post-silicon validation phase was highly successful; we were able to achieve all of our original testing objectives and exceeded our initial goals by fully validating the advanced hardware features.

The picture of the physical die is shown in Figure 6.

A. PCB

1) Bring-Up PCB Overview: We designed a custom KiCad bring-up PCB to provide a controlled post-silicon validation platform for TACO. The primary board interface is an FMC connector, allowing the chip to be driven directly from the VCU118 FPGA board for clock generation, JTAG control, GPIO stimulus, and automated checking. In addition to the FMC interface, the PCB includes 0.1-inch pin headers that expose all chip signals, enabling standalone bench testing without the FPGA carrier board.

2) Power Design: The bring-up PCB distributes a 1.8 V supply and ground to the TACO die through the package pins, with local bulk and decoupling capacitors placed near the chip supply pins to reduce supply noise during switching activity. When the board is connected through the FMC connector, power is provided by the VCU118. For standalone testing, the same supply rails can be driven externally through the header pins using a bench power supply.

3) Clocking and Pin Connectivity: The top clock and JTAG clock can be supplied in two ways: through differential clock pairs from the FMC connector, which are converted into single-ended clocks using two differential transceivers on the PCB, or directly through the pin headers as external bench clock inputs. This dual-clock-input structure makes the board useful both for integrated VCU118-based testing and for simpler standalone measurements.

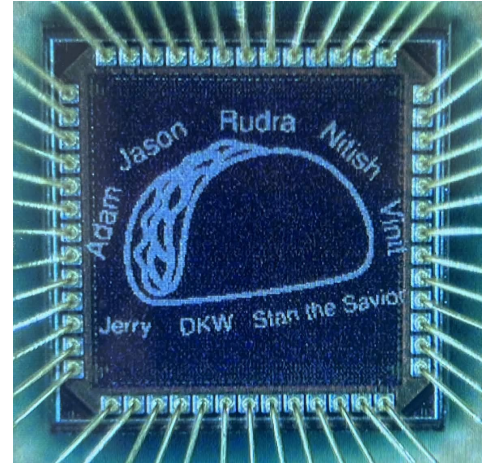


Fig. 6: Physical Die Shot

B. FPGA Host Controller

1) Host State Machine Overview: For board-level bring-up, we implemented a synthesizable host controller on the VCU118 that functions similarly to our HVL simulation testbench. The host uses the FMC-connected PCB to drive TACO reset, clocks, JTAG, and GPIOs, then programs the chip through an FSM that shifts in the clock-divider, GPIO, BRAM/JTAG, and fabric bitstream configurations. After programming, it reads back a CSR value to verify that the expected number of configuration bits was shifted in, then resets the fabric and starts functional checking.

2) Golden Model and Checker: The synthesized checker on the VCU118 acts as a hardware randomized testbench for the physical TACO chip. It generates random input stimulus using LFSRs with randomized seeds, drives those inputs into TACO through GPIO, and compares the chip outputs against the synthesized golden model. The checker reports programming completion, CSR-valid status, pass, fail, and error conditions through LEDs on the VCU118, making the bring-up result directly visible during hardware testing.

3) Automation Script: The VCU118 host design is generated using `run_rand_test_host.py`, which mirrors the simulation regression script `run_rand_test.py`. Given a target RTL design and its generated bitstream/configuration files, the script modifies the host checker code for synthesis onto the VCU118. It inserts the design's golden model, connects mapped input and output ports to the correct TACO GPIO pins, generates LFSR-based random stimulus logic, updates the output comparison logic, and writes the FPGA and GPIO bitstreams into ROM initialization files used by the host controller. This makes changing hardware test cases a smooth drop-in replacement: a new RTL design and bitstream can be swapped into the same host flow with minimal manual edits.

C. OpenOCD-Based JTAG Bring-Up

1) JTAG-HS3 Breakout Setup: OpenOCD is an open-source tool for controlling on-chip debug interfaces such as

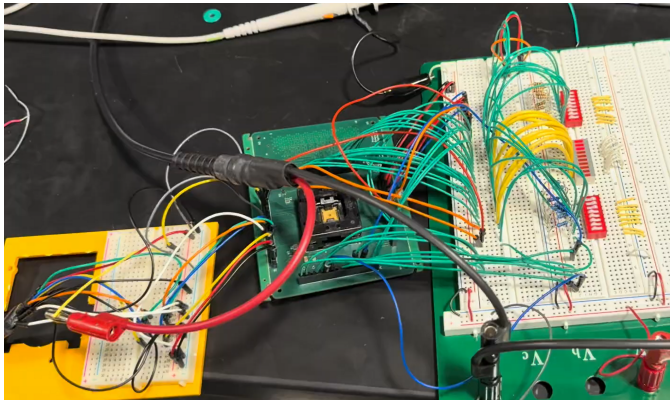


Fig. 7: Standalone Testing Setup

JTAG from a host computer. In addition to the VCU118 host controller, we also brought up TACO using OpenOCD through a Digilent JTAG-HS3 cable. The bring-up PCB exposes the TACO JTAG signals so that the HS3 can connect directly through a breakout/header path. This provides an independent programming and debug path from the FPGA-host flow, which is useful for isolating board-level wiring, TAP-controller behavior, and chip response during early bring-up.

2) **OpenOCD Configuration:** The OpenOCD flow uses a custom configuration file for the TACO TAP. The setup selects JTAG transport, uses the Digilent HS3 FTDI interface, and starts at a low adapter speed for reliable initial testing. The TACO TAP is configured with a 4-bit instruction register and the expected IDCODE 0×14951185 . This allows OpenOCD to scan the ID, identify the chip, and provide TCL commands for shifting instructions and reading/writing data through the TAP controller.

3) **Sanity Tests and Debug Flow:** Before full programming, we used basic JTAG sanity checks to validate the connection. These include resetting the TAP controller, shifting the BYPASS instruction, checking that TDO follows TDI with the expected one-bit delay, and optionally reading the IDCODE register. These tests confirm that the HS3 breakout, PCB routing, JTAG pins, and TAP state machine are functioning before attempting longer configuration shifts.

D. Summary of Results

During the bring-up phase, we executed a structured testing sequence to verify the chip's full functionality. The key results are summarized below.

a) **Initial Bring-Up and JTAG Configuration:** We initiated post-silicon validation by confirming that the chip received stable power and exhibited basic functionality. During the initial bring-up, we encountered cycle timing violations on the host FPGA that disrupted JTAG communication. After correcting these timing issues, we successfully established a reliable JTAG link. We verified this connection by reading a CSR designed to accurately monitor the total number of shifted bits.

b) **Basic Logic and I/O Debugging:** Our initial attempts to test basic AND and OR logic gates on the fabric were unsuccessful due to a slight misalignment between the VCU118 host board buttons and the TACO GPIO pin mappings. After identifying and correcting these pin mappings, we successfully demonstrated the basic logic circuits functioning correctly on the chip.

c) **Complex Logic Execution via Host FPGA:** Following the successful basic logic tests, we increased the complexity of our testing by mapping an 8-bit ALU onto the chip. In this configuration, the host FPGA provided the input vectors directly to TACO. The TACO fabric successfully executed the operations and reliably returned the correct computed outputs back to the host.

d) **OpenOCD Programming Integration:** To program the chip using OpenOCD, we utilized a JTAG HS3 cable. We initially observed incorrect behavior due to ambiguity in the cable pinout documentation. After identifying the correct pin mapping, we were able to reliably program the TACO fabric using the OpenOCD software.

e) **Multi-Chip Scalability Test:** To demonstrate the architectural scalability of our design, we conducted a two-FPGA test. One TACO chip was programmed via the VCU118 to run an AND gate, while a second TACO chip was programmed via OpenOCD to run an OR gate. We successfully passed the output of the first TACO directly as an input to the second TACO. This test proves that multiple TACO chips can theoretically be cascaded together to support larger, distributed RTL designs. A video demonstration of this multi-chip test is available to view.

f) **Standalone Operation:** With OpenOCD fully functional, we demonstrated TACO operating in a truly independent environment. Utilizing an external oscillator to provide the clock and the JTAG HS3 cable for programming, we mapped the 8-bit ALU to a physical breadboard setup. We utilized physical switches to drive the inputs and LEDs to display the outputs, as shown in Figure 7, verifying that all ALU operations executed flawlessly in this standalone configuration. A video demonstration of this standalone operation can also be viewed online.

g) **Advanced Features Validation:** We also validated the custom hardware IP blocks integrated into the fabric. For the BRAM, we successfully wrote data into the memory over the JTAG interface and confirmed that both the TACO logic and JTAG interface could correctly read and output the stored values. For the MAC units, we mapped a dedicated arithmetic circuit to the fabric and verified that TACO produced the correct outputs, confirming the functionality of the in-fabric accelerators.

h) **Maximum Frequency Testing:** To investigate the maximum frequency that TACO can support, We configured the VCU118 host FPGA to incrementally push the external clock frequency while running a simple NOT gate circuit on the TACO fabric. The chip successfully executed the logic up to 250 MHz, operating exactly as expected. Upon reviewing the board-level constraints, we realized that this 250 MHz

```
(.venv) root@83d54a093ec:~/taco/sw/flow# taco-
taco-convert-blif      taco-gen-constraints  taco-gen-pseudo-bitstream  taco-openocd-bringup
taco-flow              taco-gen-default-constraints  taco-gui                    taco-openocd-publish
```

Fig. 8: TACO Flow Python Packages

ceiling was actually the maximum rated frequency of the differential transceivers on our custom PCB, rather than the limit of the TACO chip itself. Consequently, while the silicon likely supports higher speeds, verifying the true maximum frequency of the fabric would require a redesigned PCB equipped with higher-bandwidth transceivers.

i) Custom Bitstream Generation and Programming

Toolchain: During the bring-up process, we streamlined our custom bitstream generation and programming toolchain. Verilog to Routing (VTR) performs Verilog synthesis via Yosys, logic optimization and technology mapping via ABC, and place and route using Versatile Place and Route (VPR). We modularized our toolchain via a suite of Python packages. These include packages to process outputs from VTR and generate our bitstream, packages to generate constraints files for VPR (either automatically or via TACO Design Constraint (.tdc) files) so users can control where inputs and outputs are located, and packages to program TACO with the bitstream via OpenOCD. We also provide the *taco-flow* package, which is an end-to-end bitstream generation workflow that can take in a Verilog file and a .tdc file and produce a bitstream which is ready to be programmed onto TACO. Furthermore, we provide both a CLI tool and a graphical user interface for using these packages. By using the GUI, a user can simply select a Verilog file and constraints file (optional) and press one button to generate the bitstream and another button to program TACO. Our entire toolchain setup is Dockerized (including all external dependencies) and documented so that it is replicable by others.

A list of the packages available can be seen in Figure 8, and a graphic showing the GUI can be seen in Figure 9. A video demonstration of the CLI and a video demonstration of the GUI can be viewed online.

IX. INDIVIDUAL CONTRIBUTIONS

A. Jason

- Full Testbench Verification Setup (RTL, Post-Synthesis, Post-PnR) + Python Regression Setup.
- Innovus PnR scripts.
- Virtuosso Integration (DRC + LVS).
- PCB Design + FPGA Bring-Up.

B. Rudra

- Compiler Flow + Bitstream Generation.
- Synthesis + Innovus PnR scripts.
- Virtuosso Integration (DRC + LVS).
- PCB Design + FPGA Bring-Up.

C. Nitish

- RTL Design + Preliminary Verification.
- Core Architecture + Advanced Features (MAC Integration, Clock Divider, GPIO).

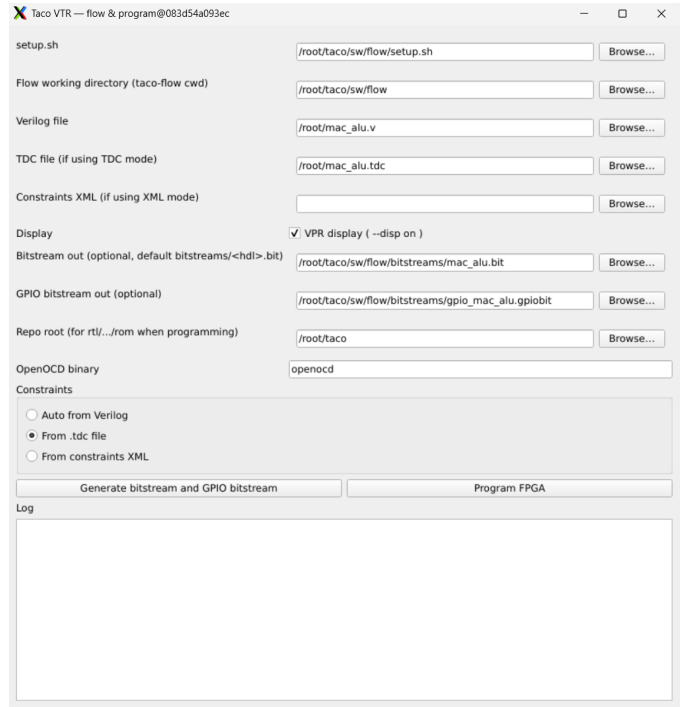


Fig. 9: TACO Flow GUI

- DRC Fixes (I/O Pad).
- FPGA Bring-Up.

D. Vinit

- RTL Design + Preliminary Verification.
- Core Architecture + Advanced Features (BRAM Integration, CSRs, GPIO).
- Debugging.
- FPGA Bring-Up.

E. Feiyang

- Compiler Toolchain.
- Synthesis Scripts + Post-Synthesis Verification + Innovus PnR Scripts.
- Virtuosso Integration (DRC + LVS).
- FPGA Bring-Up.

X. MAJOR CHALLENGES AND LESSONS LEARNED

One major challenge we encountered was that one of the larger benchmark designs mapped to our FPGA was failing when compared to the golden model. We spent considerable time debugging this issue before identifying that the VTR toolchain was not correctly processing OR gate logic. This required us to implement additional processing steps to handle OR gates properly within the VTR flow. Resolving this bug reinforced the importance of thorough, bottom-up verification and motivated our decision to develop a validation flow in which all standard cells are first verified on a 2×2 FPGA.

Another major challenge we encountered was a combinatorial loop bug that appeared during Post-PnR testbench simulations. Our initial assumption was that once the bitstream

was generated, there should be no active combinational loops within the FPGA fabric, as the VTR toolchain guarantees this. However, we discovered that an unused CLB input was being driven by its own output. This occurred because VTR assumes unused CLB inputs are undriven, whereas in our fabric, we default our CBs to track 0, which happened to be the same track driven by the CLB output. The bug only surfaced with SDF annotations, since under normal conditions, an unused input should not affect the CLB output. To address this issue, we modified our RTL design and compiler toolchain so that all unused CLB inputs are set to 0. This experience reinforced the need to be cautious when making simplifying design assumptions and highlighted the importance of writing stable logic.

REFERENCES

- [1] Xilinx Inc., “7 Series Configurable Logic Block User Guide,” UG474 (v1.7). [Online]. Available: https://www.eng.auburn.edu/~nelson/courses/elec4200/FPGA/ug474_7Series_CLB.pdf
- [2] V. Betz, J. Rose, and A. Marquardt, “Architecture and CAD for Deep-Submicron FPGAs”. Springer, 1999. [Online]. Available: <https://link.springer.com/book/10.1007/978-1-4615-5145-4>
- [3] K. Compton and S. Hauck, “Reconfigurable Computing: The Theory and Practice of FPGA-Based Computation”. ACM, 2010. [Online]. Available: <https://dl.acm.org/doi/10.5555/1564780>
- [4] University of Illinois Urbana-Champaign, “ECE 498HK: GRAPE.” [Online]. Available: <https://courses.grainger.illinois.edu/ECE498HK/fa2024/projects.html>
- [5] TinyFPGA, “TinyFPGA.” [Online]. Available: <https://tinyfpga.com/>
- [6] L. Braman *et al.*, “OpenFPGA,” GitHub repository. [Online]. Available: <https://github.com/lnis-uofu/OpenFPGA?tab=readme-ov-file>
- [7] The FABULOUS Project, “FABULOUS FPGA Toolflow,” Read the Docs. [Online]. Available: <https://fabulous.readthedocs.io/en/latest/>
- [8] AMD, “Spartan-3 FPGA Family Data Sheet,” Rev. 3.1. [Online]. Available: <https://docs.amd.com/v/u/en-US/ds099>
- [9] Xilinx Inc., “XC4000E and XC4000X Series FPGA Product Specification,” [Online]. Available: <https://www.xilinxsemi.com/datasheet/xilinxsemi/XC4036XL-1HQ160I.pdf>
- [10] YosysHQ, “Yosys Open Synthesis Suite,” GitHub repository. [Online]. Available: <https://github.com/YosysHQ/yosys>
- [11] YosysHQ, “nextpnr: Next-Generation Place and Route,” GitHub repository. [Online]. Available: <https://github.com/YosysHQ/nextpnr>
- [12] C. Wolf, “Project IceStorm: Open source flow for Lattice iCE40 FPGAs,” Read the Docs. [Online]. Available: <https://prjicestorm.readthedocs.io/en/latest/overview.html#what-is-project-icestorm>
- [13] OpenFPGA Project, “Architecture Description Language: Configuration Protocol,” Read the Docs. [Online]. Available: https://openfpga.readthedocs.io/en/master/manual/arch_lang/config_protocol/#configuration-protocol
- [14] R. Sivaswamy, G. Konjevod, and S. Hauck, “On the Feasibility of FPGA-Based Networks,” Proc. FPGA Netw. Workshop, Univ. Washington, 2007. [Online]. Available: <https://courses.cs.washington.edu/courses/cse591n/07au/papers/p21-sivaswamy.pdf>
- [15] OpenOCD Project, “OpenOCD: On-Chip Debug, In-Circuit Emulator, Flash Programmer.” [Online]. Available: <https://openocd.org/>
- [16] G. Yu, T. Cheng, B. Kettlewell, H. Liew, M. Seok, & P. Kinget (2017). “FPGA with Improved Routability and Robustness in 130nm CMOS with Open-Source CAD Targetability.” <https://arxiv.org/abs/1712.03411>